code. Keyword search does the opposite, matching literal terms while sometimes missing equivalent wording. Hybrid retrieval combines both.

Build a lexical index using SQLite FTS5, a BM25 library or Qdrant's sparse retrieval. Search both the lexical index and the vector store for each question. Then merge the result lists. Reciprocal Rank Fusion is a practical starting point: candidates appearing high in either ranking receive a combined score based on their positions. Do not directly average incompatible distance and keyword scores without normalisation.

Next, consider reranking. First retrieve perhaps twenty candidate passages cheaply. Then pass the question and each candidate through a cross-encoder or late-interaction reranker that judges relevance more closely. Send only the strongest few passages to the generator. This adds work and latency, so compare answer quality before and after the change.

Qdrant's published hybrid-search examples demonstrate dense and sparse prefetching, rank fusion and reranking. The important insight is not that every project must adopt Qdrant. It is that retrieval can be improved independently of generation. If a specific product code keeps disappearing from results, better keyword retrieval may solve the problem without changing your LLM.

Make answers show their working

A useful citation must point to evidence the system actually retrieved. Assign each passage a numbered label before building the prompt and demand answers cite those labels. Display the underlying filename, page or URL beside each label. Better still, have application code validate that every cited identifier was among the supplied passages.

Do not mistake a citation-shaped string for verification. The model may cite an excerpt that discusses the right topic but does not substantiate its claim. Test attribution manually: underline the sentence in the source that supports each important statement. If you cannot, the answer is unsupported even if its footnote looks immaculate.

Add an abstention rule: when the retrieved context does not settle a question, the assistant should state what is missing. The system can then suggest which policy, document owner or version should be checked rather than filling the gap with plausible prose.

Test retrieval, then test the answer

Reuse the held-out questions from Chapter 2. For each question, write down the source passage that should be retrieved, plus the factual answer or expected abstention. Start with twenty cases: direct facts, paraphrases, exact product codes, exceptions, outdated versions and unanswerable questions.

Run retrieval without generation first. Record whether the correct passage appears in the first five results: this is a simple *Recall@5* check. Review top-result precision as well. Next, run the model and score whether its answer is correct, grounded in the evidence and properly cited. A correct-looking answer accompanied by wrong retrieval is not a successful RAG result.

Investigate failures systematically. Missing text suggests extraction problems. A cut-off exception suggests chunking. Wrong documents suggest ranking or metadata filters. Unsupported conclusions suggest prompting or generation problems. Change one component, rerun the same test set and record whether performance improved. Avoid cherry-picked examples.

WHEN RAG SHOULD SAY "I DON'T KNOW"

Suppose your assistant has warranty documents for three laptop brands but receives a question about a fourth. A confident answer copied from a similar brand is worse than a refusal to guess. Build negative tests deliberately: ask for a policy absent from the index, a future price, an employee's private number and the terms of an expired offer. Check whether the assistant admits its limits and identifies what information would resolve the question. Retrieval scores alone cannot decide truth, and a low similarity score is not a universal abstention threshold. Calibrate decisions against labelled examples; where consequences matter, require human review. An honest gap is a useful result.

Your build: Chat with a document folder

By now you should have two repeatable commands: one to index local files and one to ask questions. Add three finishing touches. First, update the index after a document changes, ensuring outdated chunks disappear rather than lingering beside the replacement. Second, display full citation metadata so readers can open the supporting file. Third, maintain a CSV log of questions, retrieved IDs, answer quality and observed errors, excluding unnecessary personal data.

Then try a realistic demonstration: ask your assistant to compare return conditions across two official policies and identify an exception with its source. Check every sentence against the documents. If it passes, you have built something more valuable than a chatbot that sounds knowledgeable: a tool that knows where its information came from and exposes the evidence for inspection.

The underlying model has not changed. Its access to your knowledge has. That separation is what makes RAG such an effective first step toward an AI system you can genuinely call your own.